

RAPPORT DE PROJET FINAL : PIPELINE DE DONNEES DE JEU VIDEO



Introduction et Présentation des Sources de Données

L'industrie du jeu vidéo génère quotidiennement des volumes importants de données hétérogènes, réparties entre plateformes de distribution et bases de données communautaires. L'exploitation de ces flux présente un intérêt stratégique pour la personnalisation des recommandations et la segmentation des habitudes de consommation. L'objectif de ce projet est de concevoir une architecture Big Data de type Data Lakehouse respectant le modèle Medallion (Raw, Formatted, Usage). Cette infrastructure est hébergée sur un stockage objet distribué compatible S3 (MinIO) et orchestrée par Apache Airflow. Afin de bâtir un système de recommandation pertinent, le pipeline unifie et croise trois sources de données complémentaires : la richesse éditoriale d'IGDB, les indicateurs d'engagement de RAWG et l'activité réelle des utilisateurs de la plateforme Steam.

La première source, IGDB (Internet Game Database, propulsée par Twitch), constitue notre référentiel de métadonnées éditoriales stables. Requêtee via une API REST sécurisée par OAuth2, elle fournit les attributs indispensables au filtrage (genres, thèmes, éditeurs, franchises et dates de sortie). La deuxième source, RAWG, fait office de référentiel d'engagement communautaire. Accessible via une API REST, elle apporte des indicateurs dynamiques : scores Metacritic, tags collaboratifs, et données de rétention au sein des bibliothèques (statuts possédés, terminés ou abandonnés).

La troisième source, Steam (Valve), apporte la dimension comportementale (temps de jeu global, activité récente et liste des jeux possédés). Contrairement aux APIs d'IGDB ou de RAWG, l'acquisition des données Steam présente une contrainte technique majeure : l'absence d'un annuaire public d'utilisateurs. Pour constituer notre base d'apprentissage de manière représentative, un crawler basé sur un algorithme de parcours en largeur (BFS - Breadth-First Search) appliqué au graphe social de Steam a été développé.

Le crawler progresse de proche en proche : à partir d'un utilisateur initial (Seed ID), il extrait sa liste d'amis via la méthode GetFriendList de l'API Steam Web. Les identifiants découverts alimentent une file d'attente persistée sur MinIO (queue.json), tandis que les bibliothèques des profils visités sont extraites via la méthode GetOwnedGames sous forme de fichiers JSON bruts. Un registre de suivi des visites (processed_ids.json) assure l'idempotence du parcours. Cette méthode récursive permet de modéliser des communautés d'amis partageant des affinités réelles, fournissant ainsi la matière première nécessaire à notre modèle de clustering.

Architecture Globale & Stack Technique

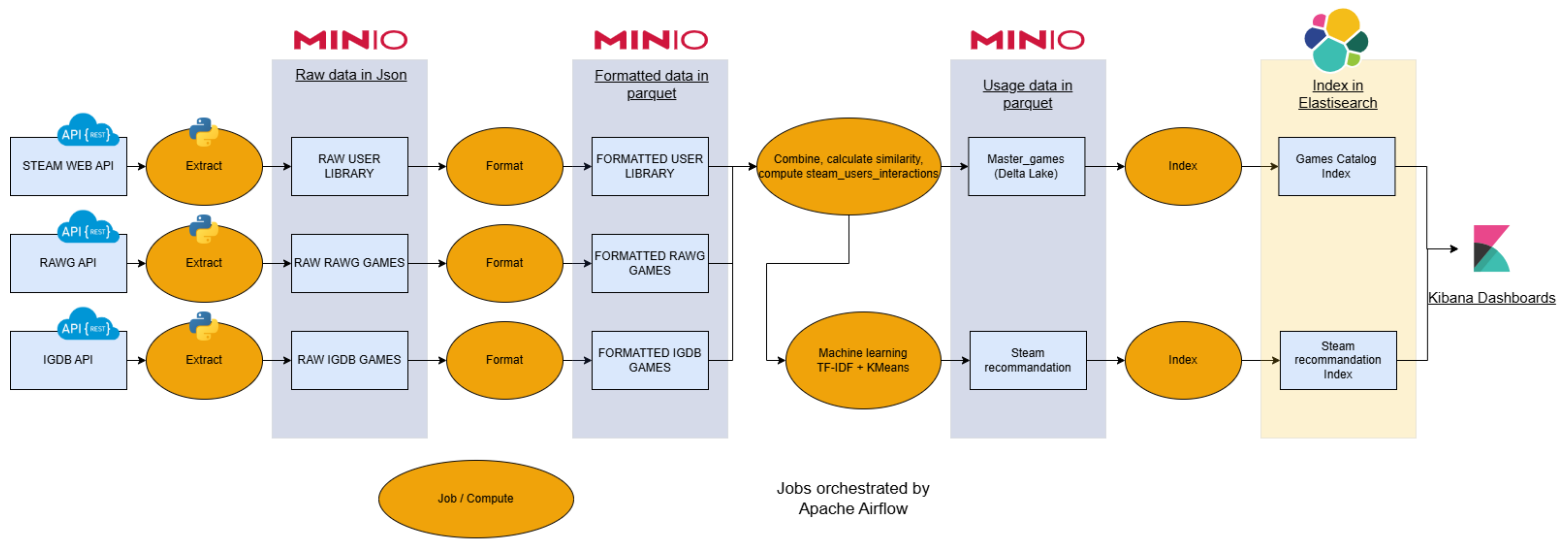


Figure 1 : Diagramme des flux de données et architecture Medallion de la plateforme gaming

Afin de soutenir ces objectifs, j'ai mis en place une architecture de données structurée en trois couches successives de raffinement, exploitant des conteneurs Docker pour garantir la portabilité de l'infrastructure.

La pile technique s'articule autour des composants suivants :

- Orchestration : Apache Airflow (v2.9.2) planifie et supervise les pipelines d'ingestion et délègue les traitements volumineux au moteur de calcul.
- Stockage (Data Lake) : MinIO fait office de stockage objet compatible S3. Il structure le stockage en trois zones (Raw, Formatted, Usage) pour assurer le découplage entre le stockage et le calcul.
- Calcul distribué : Apache Spark (v3.5.1) effectue les transformations, les jointures complexes et l'apprentissage automatique. L'extension Delta Lake (v3.1.0) apporte un support transactionnel (transactions ACID) pour fiabiliser la couche de sortie.
- Indexation et Restitution : Elasticsearch (v8.17.2) indexe les documents consolidés pour garantir des temps de réponse faibles lors des recherches. Kibana (v8.17.2) sert d'interface de visualisation pour l'analyse des indicateurs et le profilage des segments.

Le cycle de vie de la donnée se décompose ainsi : la couche *Raw* conserve les fichiers JSON bruts de manière immuable. La couche *Formatted* applique des schémas stricts et convertit les données au format colonnaire Parquet compressé. Enfin, la couche *Usage* héberge les tables consolidées, notamment le catalogue unifié géré via Delta Lake, assurant des écritures idempotentes.

Ingestion des Données & Stockage (Couche Raw)

La collecte des données issues d'IGDB, RAWG et Steam combine un traitement de masse (Bulk Ingest) pour l'initialisation, et un traitement incrémental (Daily Ingest) pour la mise à jour :

1. Ingestion de Masse (Bulk Ingestion) :

- Bulk IGDB : Après authentification, l'importation cible les 30 000 jeux les plus populaires par pagination (fenêtres de 500 enregistrements, pause de 0,4s pour respecter les quotas de l'API). L'état d'avancement est sauvegardé sur MinIO (initial_import_state.json) pour tolérer d'éventuelles interruptions.
- Bulk RAWG : Ce processus extrait les 100 premières pages (soit 4 000 jeux) ordonnées par popularité, avec un suivi d'état similaire.

2. Ingestion Journalière et Continue :

- Daily IGDB & RAWG : Ces tâches quotidiennes filtrent les enregistrements modifiés au cours des dernières 24 heures (via le champ updated_at pour IGDB et le paramètre updated pour RAWG). Les scripts intègrent une gestion des limites de requêtes (HTTP 429) avec pause adaptative.
- Steam BFS Crawler : Ce script de scraping récursif s'exécute à intervalles réguliers. Pour respecter les quotas de l'API Steam, un délai aléatoire (*jitter* entre 0,25s et 0,4s) est appliqué entre les appels. En cas de détection d'une erreur HTTP 429, le crawler sauvegarde son état courant sur MinIO et lève une exception Airflow, suspendant temporairement la tâche pour une durée de 15 minutes.

Les fichiers bruts sont structurés dans le bucket de stockage selon le modèle suivant : data/raw/<group>/<dataEntity>/<dateVersion>/.

Normalisation Spark & Validation de Schéma (Couche Formatted)

Les jobs de normalisation Spark nettoient et structurent les fichiers JSON bruts pour les transcrire au format Parquet. Cette transformation repose sur plusieurs principes :

- Schémas Stricts : L'application d'un schéma explicite lors de la lecture des JSON bruts évite les dérives de types et assure la présence des colonnes indispensables, même si certaines valeurs sont initialement absentes.
- Normalisation Temporelle : Les timestamps Unix d'IGDB sont convertis en types temporels SQL standardisés en UTC. Les dates de RAWG, initialement au format texte ISO, sont transtypées en types DateType ou TimestampType.

- **Aplatissement des Structures Imbriquées** : Les tableaux d'objets complexes sont simplifiés. Par exemple, le tableau de genres d'IGDB est réduit à une liste à plat de chaînes de caractères (genres.name devient genre_names). De même, les entreprises partenaires sont filtrées pour distinguer clairement les développeurs des éditeurs.
- **Gestion des Valeurs Manquantes et Consolidation** : Les chaînes nulles sont remplacées par des valeurs vides et les évaluations manquantes par 0. Pour la consolidation des profils Steam, le script de formatage limite le traitement aux joueurs possédant moins de 500 jeux. Ce seuil permet d'exclure les bibliothèques anormalement volumineuses (souvent associées à des robots ou à des comptes de collectionneurs), qui provoquaient des surcharges mémoires lors des phases de traitement distribué.

Fusion des Données & Similarité Algorithmique (Couche Usage - Catalogue)

La couche *Usage* croise les sources d'informations afin de produire de la valeur analytique. Le catalogue de référence (master_games) est stocké au format Delta Lake.

1. Combinaison des Sources :

La jointure s'effectue en deux étapes. Les catalogues IGDB et Steam partageant le champ SteamAppId, leur association est directe. En revanche, l'API RAWG ne disposant pas de cette clé, j'ai implémenté une jointure sémantique basée sur le nom normalisé du jeu (nettoyé des espaces et des caractères spéciaux par expression régulière) associé à l'année de sortie, évitant ainsi les collisions entre reboots homonymes.

2. Calcul de la Similarité de Jaccard :

Pour calculer les proximités entre jeux sans réaliser de produit cartésien complet (très coûteux en mémoire), l'approche suivante a été retenue :

- **Regroupement des tags** : Fusion des genres, thèmes et tags sous un vecteur unique (unified_tags).
- **Index Inversé** : Explosion du vecteur pour obtenir un couple (Jeu, Tag).
- **Filtrage du bruit** : Exclusion des mots-clés trop génériques apparaissant sur plus de 1 500 titres (ex. : "action", "solo").
- **Auto-jointure interne** : Association des jeux partageant au moins un tag significatif commun.
- **Indice de Jaccard** : Calcul du score de proximité sur les paires filtrées :

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

Une fonction de fenêtrage extrait ensuite les 10 meilleures recommandations pour chaque titre.

L'écriture finale s'effectue via une opération MERGE Delta Lake, garantissant l'idempotence du pipeline.

Moteur de Recommandation & Machine Learning (Couche Usage - ML)

Le second livrable de la couche *Usage* est un moteur de recommandation personnalisé pour les joueurs de Steam (steam_recommendations), combinant approche thématique et filtrage par cohorte.

1. Vectorisation (TF-IDF) & Clustering (K-Means) :

- Pour chaque utilisateur, l'ensemble des tags des jeux auxquels il a joué est agrégé pour former un profil d'intérêt global.
- Un modèle CountVectorizer extrait la fréquence de ces termes (en ignorant ceux présents dans moins de 5 profils), suivi d'une pondération IDF pour minimiser l'impact des termes très fréquents. Les vecteurs sont normalisés (L2) pour simuler une distance de cosinus.
- Un modèle K-Means est entraîné sur ces données avec un paramètre fixé à $K = 30$ clusters, déterminé comme pertinent suite à l'analyse du score de silhouette. Chaque joueur est ainsi affecté à un cluster représentant ses préférences majeures.

2. Profilage Dynamique par le calcul du Lift :

Pour expliciter la thématique de chaque cluster sans intervention humaine, le système calcule le Lift de chaque tag :

$$\text{Lift} = \frac{\text{Proportion du tag dans le cluster}}{\text{Proportion du tag globalement}}$$

Les deux tags ayant le Lift le plus élevé (et exclus d'une liste de mots génériques) servent à nommer la cohorte (par exemple : Cluster 3 (RTS & Tactical Fans)).

Filtrage de Popularité et de Possession :

Afin de proposer des recommandations pertinentes et d'éviter de suggérer uniquement des titres omniprésents, un score pondéré est appliqué :

$$\text{game_cluster_score} = \frac{\text{cluster_score}^{1.2}}{\ln(1 + \text{global_score})}$$

Les jeux détenus par plus de 20 % de la population globale ou déjà possédés par l'utilisateur sont exclus, permettant de retenir les 10 suggestions affichant le meilleur score d'affinité au sein du cluster.

Indexation Elasticsearch & Préparation des Données

Une fois les calculs de la couche *Usage* finalisés sous Spark, les données sont indexées dans Elasticsearch afin de servir de source de données aux tableaux de bord Kibana.

1. Optimisations de la Structure des Données (Côté Spark)

Pour optimiser les performances d'indexation et faciliter la création des visualisations, la modélisation a été adaptée en amont pour éviter les contraintes d'Elasticsearch :

- **Transtypage des Identifiants** : Les clés primaires numériques (`igdb_id` et `steam_id`) sont converties en `StringType` sous Spark. Cela garantit qu'Elasticsearch les utilise directement comme identifiants de documents (`_id`), évitant les conflits de mapping.
- **Aplatissement des Structures Imbriquées** : Les objets imbriqués complexes (genres, tags, plateformes) sont transformés en listes de chaînes de caractères plates (`ArrayType(StringType())`). Cette dénormalisation simplifie la configuration des filtres interactifs dans Kibana et accélère le calcul des agrégations.

2. Stratégie d'Indexation et Idempotence (Upsert)

L'indexation s'appuie sur le connecteur `elasticsearch-spark-30`. Pour garantir l'idempotence des traitements (absence de doublons en cas de réexécution du pipeline), les options d'écriture ont été configurées spécifiquement :

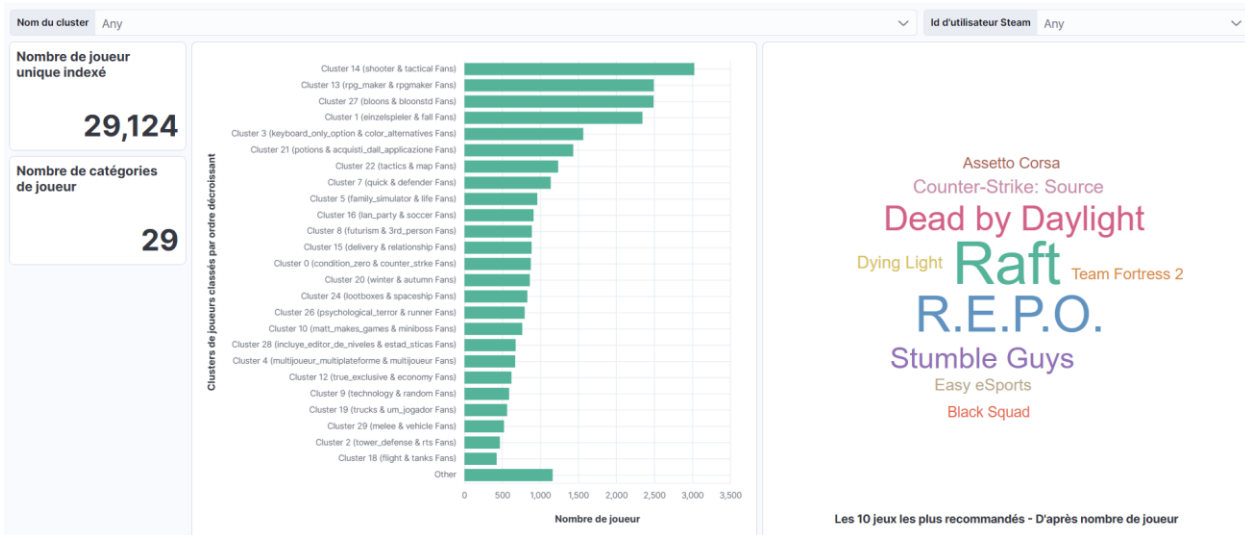
- **Gestion des ID uniques** : L'option `es.mapping.id` est associée aux clés métiers (`igdb_id` ou `steam_id`).
- **Mode d'écriture Upsert** : Avec `es.write.operation` défini sur `upsert`, Elasticsearch met à jour le document existant s'il trouve un ID identique, ou l'insère dans le cas contraire.
- **Modes de Sauvegarde Spark** :
 - **Catalogue de jeux (`games_catalog`)** : Sauvegarde en mode `append` pour intégrer quotidiennement les seuls deltas de mise à jour des jeux.
 - **Recommandations (`steam_recommendations`)** : Sauvegarde en mode `overwrite` pour remplacer intégralement l'index à chaque nouvel entraînement hebdomadaire du modèle de Machine Learning.

Tableaux de Bord Kibana & Visualisations

La restitution visuelle s'appuie sur Kibana 8.17.2 via deux Data Views connectées à nos index Elasticsearch : steam_recommendations (prédictions du modèle ML) et games_catalog (base de données unifiée). L'interface est divisée en deux vues interactives et complémentaires.

1. Vue Macro : Analyse d'Audience et des Cohortes (Index steam_recommendations)

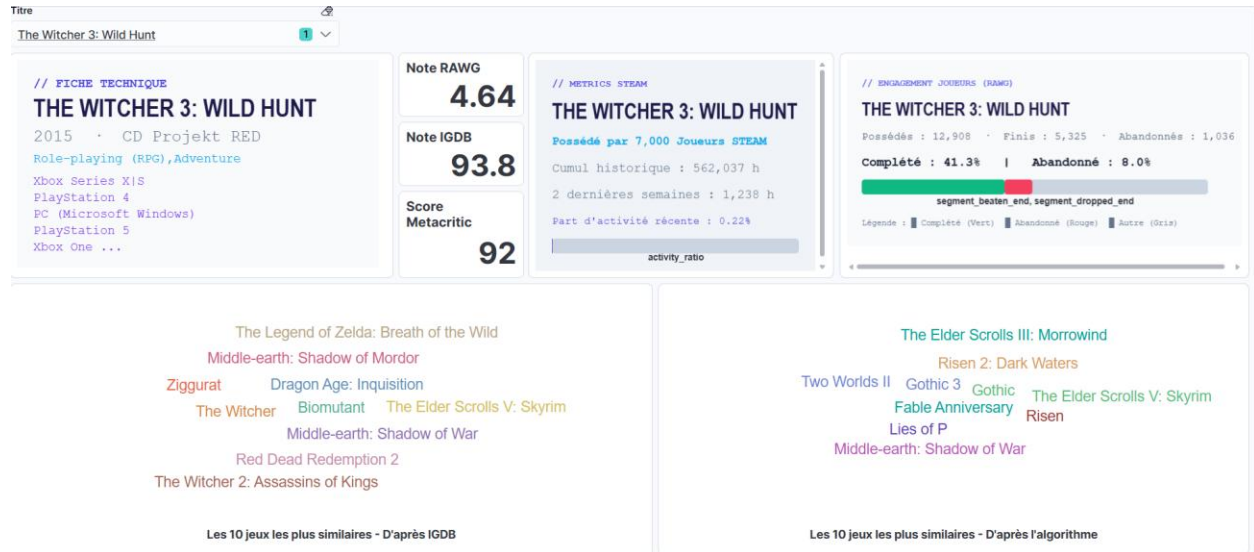
Destinée à l'analyse d'audience et aux équipes marketing, cette vue permet d'étudier la répartition des profils de joueurs et les tendances de recommandation globales.



- KPIs globaux : Présentation du nombre d'utilisateurs uniques indexés (29 124) répartis au sein de 29 catégories identifiées (l'un des 30 clusters attendus n'ayant pas réuni d'utilisateurs éligibles).
- Répartition des profils : Un graphique en barres horizontales détaille les effectifs de chaque cluster, affichant les dénominations dynamiques basées sur le Lift.
- Tendances : Un nuage de mots-clés met en évidence les 10 jeux les plus recommandés pour l'échantillon ou le cluster sélectionné.

2. Vue Micro : Fiche d'Identité et Focus Jeu (Index games_catalog)

Cette vue propose une fiche technique détaillée croisant les informations des trois sources de données (IGDB, RAWG, Steam) autour d'un jeu sélectionné, ici illustré avec The Witcher 3 : Wild Hunt, un jeu possédé par un grand nombre de personne et très populaire.



- Métadonnées et Notes : Synthèse éditoriale du titre (studio, date, plateformes) associée aux évaluations croisées (RAWG, IGDB et Metacritic).
- Engagement : Restitution du volume de joueurs Steam issus de notre échantillon, de leur volume d'heures cumulées et de leur activité récente. Un graphique RAWG illustre les taux de complétion et d'abandon déclarés par la communauté.
- Validation des Similarités : Deux nuages de mots comparent les jeux similaires éditoriaux (IGDB) et algorithmiques (calculés via l'indice de Jaccard). Cette comparaison valide la cohérence de l'algorithme, qui propose des alternatives sémantiquement proches et absentes des métadonnées d'origine d'IGDB.

Cette confrontation directe valide la pertinence du modèle en retrouvant les références du genre tout en proposant des alternatives calculées cohérentes et absentes des données d'origine d'IGDB.

Robustesse du Système & Gestion des Erreurs

L'exploitation en production d'un pipeline unifiant des APIs tierces et des tâches de Machine Learning requiert des mécanismes de résilience à chaque étape :

- Tolérance aux pannes réseau et limites de quotas : L'ingestion initiale utilise un système de suivi d'état stocké sur MinIO, facilitant une reprise après interruption. Pour le crawler Steam, la file d'attente et l'historique des visites sont persistés sur MinIO à chaque batch, et la levée d'une exception Airflow planifie une pause automatique de 15 minutes en cas de code HTTP 429.
- Transactions et Intégrité : L'implémentation de Delta Lake pour la table master_games garantit des transactions ACID. En cas d'interruption du calcul des similarités de Jaccard, un mécanisme de rollback automatique prévient l'écriture de données fragmentées ou corrompues. Le contrôle de schéma empêche également toute modification structurelle inattendue des sources d'entrée.

- Optimisation de la mémoire distribuée : Pour éviter les erreurs de saturation mémoire (*Out Of Memory*) sous Spark, les bibliothèques d'utilisateurs de plus de 500 jeux sont écartées, limitant ainsi les phénomènes de déséquilibre de charge (*data skew*) lors des jointures. Les traitements lourds s'appuient sur un dimensionnement adapté de la mémoire du Driver Spark (14 Go) et sur une augmentation à 800 du nombre de partitions de shuffle.

Limites du Système Actuel & Évolutions Envisagées

Bien que l'architecture mise en œuvre offre un socle fonctionnel robuste et réponde aux exigences de traitement de bout en bout, plusieurs limites techniques et méthodologiques ont été identifiées. Leur résolution constitue autant d'axes d'évolution pour accroître la précision du système, la qualité des données et la scalabilité globale.

1. Transition vers un modèle de recommandation individuel (Finesse algorithmique)

Limite actuelle : Le modèle de recommandation s'appuie sur une logique de cohorte (recommandation par cluster). Bien que les recommandations soient filtrées en excluant les jeux déjà possédés par l'utilisateur, deux joueurs appartenant au même cluster se voient proposer la même liste ordonnée de suggestions. Le niveau de personnalisation reste donc collectif plutôt qu'individuel.

Évolution proposée : Pour affiner les prédictions, il serait pertinent d'implémenter un algorithme de filtrage collaboratif individualisé, tel que la factorisation de matrice par moindres carrés alternés (ALS - Alternating Least Squares), disponible dans Spark ML. Ce modèle exploiterait directement le score continu de préférence implicite pour prédire une note individualisée pour chaque couple (joueur, jeu). Une approche hybride combinant le clustering actuel (pour définir le profil thématique global) et le modèle ALS permettrait d'obtenir des suggestions hautement personnalisées tout en limitant le problème du démarrage à froid.

2. Optimisation de l'exclusion des tags non anglophones (Qualité de la donnée)

Limite actuelle : Bien qu'un filtre linguistique soit appliqué sur la source RAWG (`x.language = 'eng'`), des incohérences subsistent dans les faits. La plateforme RAWG reposant sur des contributions communautaires ouvertes, de nombreux tags rédigés dans d'autres langues (italien, allemand, russe, etc.) sont erronément catégorisés comme anglophones par les utilisateurs. Ces tags polluent la phase de vectorisation et d'entraînement du modèle.

Évolution proposée : Afin d'assainir la couche *Formatted*, il serait intéressant d'intégrer une étape de validation linguistique automatisée sous Spark. Cela pourrait se faire via l'implémentation d'une fonction utilisateur (UDF) exploitant une bibliothèque de détection de langue (telle que `fasttext` ou `langdetect`), ou par le croisement des tags avec un dictionnaire de référence des termes de l'industrie du jeu vidéo en anglais. Les tags non validés seraient ainsi systématiquement écartés avant la phase d'apprentissage.

3. Intégration des métadonnées techniques dans le calcul de similarité

Limite actuelle : Le calcul de similarité entre les jeux (indice de Jaccard) repose exclusivement sur la superposition des genres, des thèmes et des tags. Cette approche purement sémantique peut rapprocher des jeux aux caractéristiques techniques ou historiques totalement divergentes (par exemple, associer un jeu de rôle rétro en 2D de 1995 et un RPG moderne en monde ouvert en 3D de 2023, sur la seule base de leurs étiquettes communes).

Évolution proposée : Pour accroître la cohérence des suggestions, le calcul de similarité gagnerait à devenir hybride et pondéré. Nous pourrions définir une matrice de distance combinant plusieurs facettes de métadonnées :

- La similarité sémantique (Jaccard sur les tags, 50 % du score) ;
 - La proximité temporelle (pénalisation de l'écart d'années de sortie, 15 % du score) ;
 - L'identité technique (moteur de jeu commun, perspective de caméra identique, 20 % du score) ;
 - L'identité éditoriale (développeur ou franchise commune, 15 % du score).
- Un tel modèle hybride permettrait de s'assurer que les recommandations respectent non seulement le thème, mais aussi l'expérience visuelle et technique recherchée par l'utilisateur.

Conclusion

En conclusion, ce projet a permis de concevoir et de valider une architecture Data Lakehouse complète, démontrant la viabilité technique de la pile Apache Airflow, MinIO, Apache Spark et Elasticsearch pour le traitement de données à grande échelle.

Au-delà de la mise en place d'un pipeline d'ingestion robuste, la valeur de ce travail réside dans le croisement de sources hétérogènes. L'association de métadonnées éditoriales froides (IGDB/RAWG) et de données comportementales réelles (Steam) a permis de concevoir un système de recommandation hybride pertinent, dont la cohérence a pu être validée visuellement sur nos tableaux de bord Kibana.

Ce travail a également mis en lumière les réalités pratiques de l'ingénierie des données, notamment la gestion des limites de requêtes des APIs externes, la nécessité d'une modélisation transactionnelle via Delta Lake pour garantir l'intégrité des calculs, et l'importance de l'optimisation des ressources Spark face aux risques de saturation de la mémoire. Bien que perfectible, notamment à travers l'implémentation de modèles de recommandation individuels ce prototype fonctionnel constitue une base évolutive et conforme aux standards modernes des architectures Big Data.